MASSIVE Диалог

Голосовое управление умным домом для незрячих пользователей

Уточнение команд и подтверждение выполнения



Подготовил Иванов Максим Сергеевич
Группа P4155 · ИСУ 564789

Выбранный датасет — MASSIVE

Amazon Science · размеченные текстовые команды голосовым помощникам

16 521 русских фраз

18 сценариев

60 намерений

55 типов параметров

Из чего состоит запись

`id: 3759 · partition: train`

utt — фраза

«разбуди меня в семь утра»

annot_utt — разметка

`разбуди меня в [time : семь утра]`

scenario — сценарий: **alarm**

intent — намерение: **alarm_set**

Зачем нужен в проекте

Фразы и intent

Обучение классификатора намерений.

Разметка параметров

Типы и примеры значений для извлечения.

train / dev / test

Обучение, промежуточная и итоговая проверка.

Правила уточнения, согласования и исполнения реализованы в приложении.

MASSIVE — источник данных. **Диалог** — уточнение команды и подтверждение результата.

Источник: Amazon Science / MASSIVE, CC BY 4.0. Числа относятся к локальному massive_ru.csv; после очистки — 15 642 фразы.

ETL: из CSV в PostgreSQL

Стандартные csv, re, dataclasses и Counter · SQLAlchemy

Извлечение
16 521 строка CSV
6 обязательных колонок

→ **Проверка и очистка**
Пустые поля, ID, partition, дубликаты и разметка

→ **Преобразование**
Пробелы, справочники, типы параметров

→ **Загрузка через ORM**
Сущности и связи в реляционной БД

16 521 исходных строк

− **879** повторяющихся фраз

= **15 642** фразы после очистки

Повторный импорт безопасен

external_id уже есть в БД → запись пропускается.

Справочники ищутся по уникальному коду. Результат и причины отклонения записываются в лог.

Данные и правила разделены

Фразы и разметка — MASSIVE.

Обязательные параметры, порядок и вопросы — правила приложения в labels.py.

```
python -m app.etl.import_massive --source data/massive_ru.csv
```

Код: app/etl/import_massive.py и app/etl/labels.py. На этом CSV все 879 отклонений — дубликаты; другие проверки также реализованы.

Модель прогнозирует намерение

scikit-learn: Pipeline + FeatureUnion + CalibratedClassifierCV

Текст команды
«поставь будильник через минуту»

→ **TF-IDF**
Слова: n-граммы 1–2
Символы: 2–5

→ **LinearSVC**
Веса для 60 классов
Калибровка: cv=3

→ **Прогноз**
intent: alarm_set
confidence: 0.896

Обучение и проверка разделены

train: 10 907 — fit модели и словарей.
dev: 1 916 — промежуточная оценка.
test: 2 819 — итоговая оценка.

Параметры извлекаются отдельно

Словарь значений из train + правила времени и даты.

Пример: `time = «через минуту»`.

confidence оценивает класс, а не успех действия.

82,8%
accuracy на test

0,7859
macro-F1 на test

Оценка базового текстового классификатора.
Это не точность ASR и не доля исполненных команд.

Код: app/services/nlu.py · app/evaluate.py · docs/evaluation.json. Pipeline хранится в памяти процесса; обучение при первой команде или POST /api/nlu/train. Для явных включить/выключить свет добавлено узкое правило.

Как бэкенд обрабатывает команду

Микрофон → ASR (Vosk / браузер) → текст в HTTP-запросе → JSON-ответ → TTS

FastAPI → **Pydantic** → **Сервисы** → **ORM и запросы** → **PostgreSQL**

api/
маршрут и HTTP

schemas/
валидация и DTO

services/
NLU и диалог

repositories/, models/
SQLAlchemy Session

Команды, параметры,
состояния и квитанции

Запрос

```
POST /api/dialogues
{"text": "поставь будильник"}
```

201 — создание · 400 — бизнес-ошибка
404 — нет объекта · 409 — конфликт · 422 — валидация

Ответ 201 · фрагмент

```
{
  "intent": "alarm_set",
  "phase": "clarifying",
  "question": "На какое время поставить будильник?",
  "execution_confirmed": false
}
```

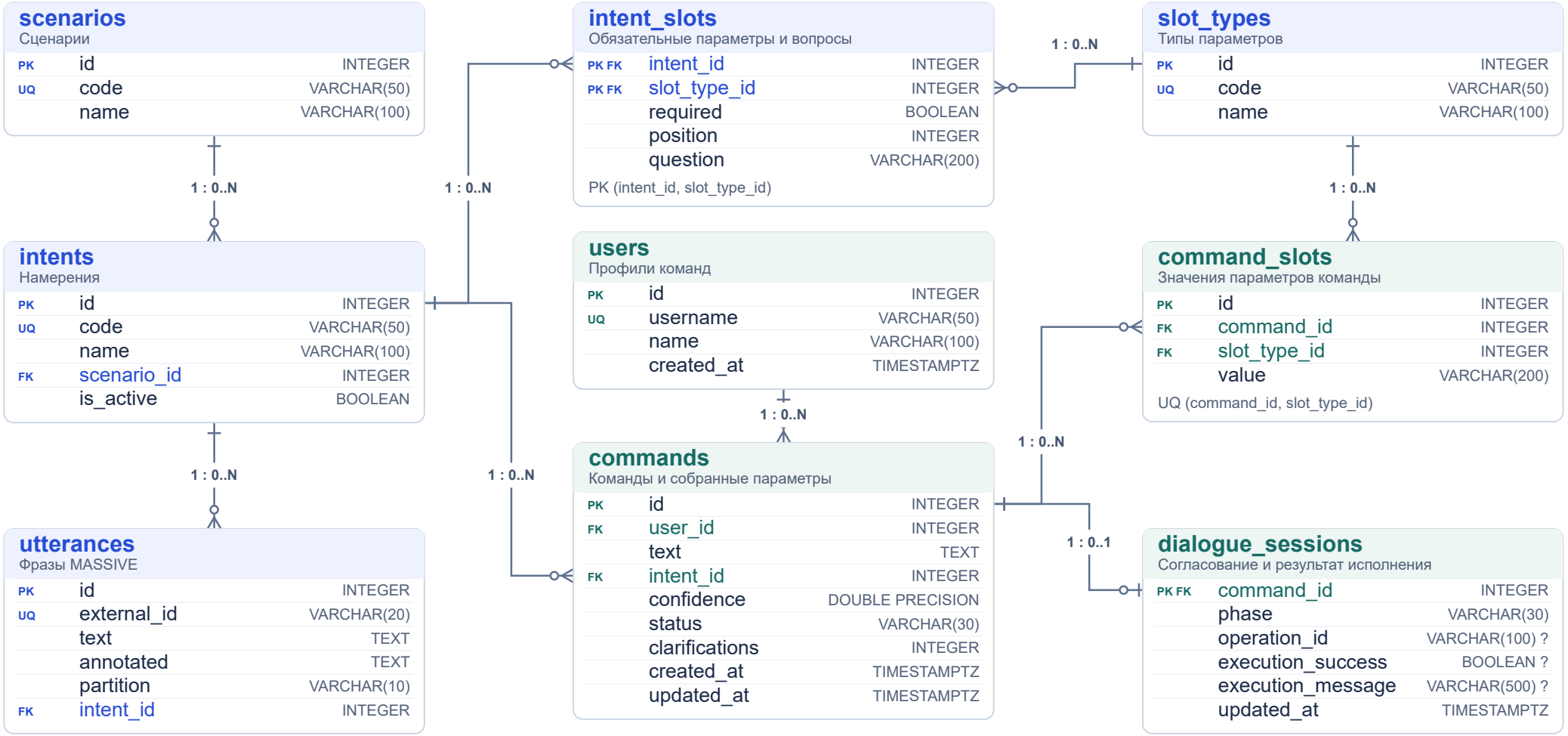
`clarifying` → `awaiting_confirmation` → `ready` → `executed` / `execution_failed`

Обязательные параметры → проверка формата → согласие → квитанция исполнения.

/answer продолжает диалог, /execution принимает результат. Квитанция: блокировка строки и идемпотентный повтор. Весь диалог — несколько транзакций. CRUD справочников, фильтры и единый формат ошибок сохранены.

9 связанных сущностей

45 атрибутов · 9 внешних ключей · связи 1:N и M:N



PK — первичный ключ · FK — внешний ключ · UQ — уникальность · ? — допускается NULL · M:N — через intent_slots и command_slots

M:N: intent_slots и command_slots. **Целостность:** PK, FK, UNIQUE, CHECK и индексы. **Схема:** обратимые Alembic-миграции; bootstrap и seed для запуска.

Диаграмма построена из ORM-моделей: app/models/*.py и scripts/render_erd.py. Квитанция хранится в dialogue_sessions; commands.status=completed означает собранные параметры.

Тесты проверяют поведение бэкенда

146
pytest-тестов

92,37%
покрытие Python-кода

16
frontend-тестов

Что покрываем

- ETL: очистка, разметка, повторный импорт.
- API: CRUD, фильтры, пагинация и ошибки.
- Диалог: параметры, лимит, согласие и квитанции.
- БД: ограничения, связи, история и агрегации.

Как изолируем проверки

pytest + FastAPI TestClient + pytest-cov.

SQLite в памяти — отдельно от рабочей БД.
Миграции и запуск дополнительно проверяются на PostgreSQL.

Порог покрытия в CI — 70%.

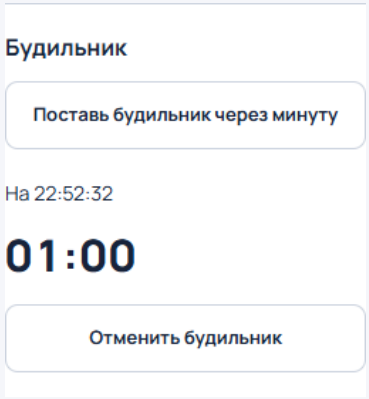
push / PR → **Параллельные jobs**
lint · test · frontend · migrations → **Docker**
сборка · bootstrap · healthcheck · API

Git: отдельные ветки и Pull Request. CI автоматизирован; автоматического развёртывания в облаке нет.

Код: tests/ · frontend/test/ · .github/workflows/ci.yml. GitHub Actions проверяет зависимости, Ruff, Prettier, тесты, миграции и контейнер. Взаимное code review по методичке — отдельное требование.

Три сценария с результатом в браузере

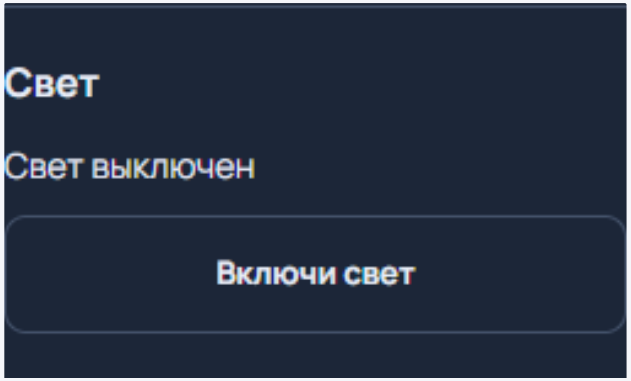
Будильник



«Поставь будильник через минуту»

Отсчёт и звуковой сигнал

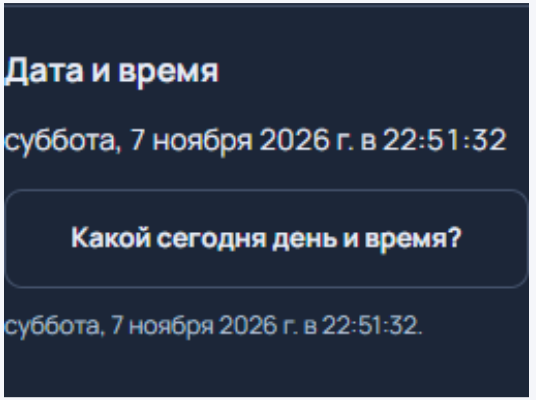
Свет



«Выключи / включи свет»

Меняется фон страницы

Дата и время



«Какой сегодня день и время?»

Ответ по часам устройства

```
{
  "phase": "executed",
  "execution_confirmed": true,
  "execution_message": "Свет выключен."
}
```

После согласия адаптер выполняет действие и отправляет POST /api/dialogues/{id}/execution. История и аналитика читают сохранённый результат из БД.

Скриншоты работающего приложения. Для будильника подтверждается установка; для сигнала вкладка должна оставаться открытой. Физические лампы и системный будильник не подключены.

Промежуточный ассистивный сервис

Ограничения текущей версии

- 60 известных намерений; возможны ошибки на новых формулировках.
- Слоты и даты ограничены словарями и правилами.
- Исполнители работают в браузере; физические устройства не подключены.
- Выбранный профиль, без идентификации голоса. Нет пользовательской авторизации и испытаний с незрячими участниками.

Подходы в реальных системах

Шаблоны и intent

Home Assistant: команды, параметры и реальные сущности дома.

Transformer-NLU

Предобученные модели для намерений и извлечения сущностей, например в Rasa.

LLM + инструменты

Модель формирует вызов; приложение проверяет и исполняет его.

Для реальной интеграции

Авторизованный адаптер → API устройства → проверенный результат → квитанция.

Нормализация времени · права доступа · таймауты · защита от повторного действия

Более сложная модель не заменяет согласование и проверку исполнения.

Примеры подходов: Home Assistant: voice architecture · Rasa: NLU components. Это варианты развития, не реализованные интеграции проекта.

От датасета — к Python-информационной системе

01 **Данные**

Очистка, идемпотентный ETL, реляционная модель и аналитика.

02 **Бэкенд**

FastAPI, Pydantic, SQLAlchemy, классификатор и бизнес-правила диалога.

03 **Проверяемый результат**

Квитанции исполнения, тесты, GitHub CI и запуск через Docker Compose.

Уточнить → согласовать → выполнить → озвучить результат

Следующий шаг: адаптер реального устройства и проверка доступности с пользователями.

Репозиторий: github.com/MaksChocomint/massive-assistant-project. Приложение и Swagger: localhost:8010 · /docs.